University of Puerto Rico
Campus of Mayagüez
Mayagüez, Puerto Rico
Department of Electrical and Computer Engineering

ARTIFICIAL INTELLIGENCE
Technical Report: Group H
Is Python Fast or Slow?

By

Claudia Sánchez Merino
Manuel A. Infante Degro
Carlos M. Chamochumbi Canals
Emmanuel A. Morales Martínez

Prof. José F. Vega Riveros
ICOM5015
25 / February / 2026

Purpose of the assignment

The purpose of this assignment is to investigate and compare the computational performance of matrix multiplication implemented using direct Python code versus NumPy's mathematical libraries. Matrix multiplication is a fundamental operation for machine learning (ML) algorithms primarily due to their reliance on repeated linear algebra computations for their training [1]. As is the case in mobile edge computing (MEC), matrix multiplications become the dominant computational cost for a majority of neural network architectures and machine learning applications, which is why it is significant to find ways of accelerating these calculations and reduce their computational cost [1]. Although Python allows matrix multiplications through direct implementation using nested loops, this approach is limited by the capabilities of the interpreter and the language, which can reflect similar issues in calculation performance as in current model training. However, compiled libraries that can delegate the heavy numerical operations like NumPy can shed light into how computational calculations can be improved, resulting in more efficient multiplications at a larger scale. In this instance, the complexities of the multiplications are done at a smaller scale than regular ML applications, but modest improvements in calculations can reflect possibilities to improve overall performance for matrix multiplications.

Key Hypothesis

The primary hypothesis for this experiment is that NumPy's implementation of matrix multiplication using its math libraries will demostrate faster execution times than Python's nested loop implementation. It is expected that the difference in performance will be more pronounced as more elements are introduced in 1-D arrays and matrix sizes increase in 2-D multiplications. This hypothesis is based on the architectural difference between the two approaches. Python's nested loop approach relies on interpreter-level execution, where each iteration in the nested loop is counted individually, causing longer runtime evaluations as operations increase. Meanwhile NumPy's approach uses optimized built-in functions that handle large calculations efficiently with reduced strain on performance.

Experimental design choices

The comparison in calculations between direct Python code and NumPy's libraries will be done through the use of nested loops versus mathematical functions. Both the direct Python implementation and the math library implementation will calculate element-wise multiplications of (1-D) arrays and true matrix multiplications with (2-D) arrays. 1-D arrays will utilize between 10, 50, 100, 200 and 500 elements, while 2-D arrays will calculate 10x10, 20x20, 50x50, 100x100, 200x200 and 500x500 matrices respectively.

```python
def python_vector_multiply(a,b):
  result = [0.0] * len(a)
  for i in range(len(a)):
    result[i] = a[i] * b[i]
  return result

def python_matrix_multiply(A,B):
  n = len(A)
  result = [[0.0] * n for _ in range(n)]
  for i in range(n):
    for j in range(n):
      sum_ij = 0.0
      for k in range(n):
        sum_ij += A[i][k] * B[k][j]
      result[i][j] = sum_ij
  return result
```

Figure 1: Code implementation for Nested Loops

The python program will run these defined functions found in figure 1, where each matrix multiplication type is a loop. 1-D arrays are computed element by element, while 2-D multiplication requires the use of nested loops to calculate the product of each row and column of the matrices.. The program will call these functions and return the calculation times for each attempt in size.

```python
(lambda x,y: x*y, a, b) (lambda x,y: x@y, A, B)
```

Figure 2: Code implementation for NumPy

The mathematical functions used for Numpy's approach are the multiplications a*b and A@B. A and B are values passed as NumPy arrays or NumPy matrices, which allows for direct multiplication due to an internal multiplication routine coded into NumPy. Lambda x,y is used as a temporary function wrapper so the NumPy operation can be passed into the timing function of the multiplication.

Measurement methodology

Execution time was measured using time.perf_counter, with all input vectors and matrices generated using a fixed random seed and stored as float64 values. Data creation was performed outside the timed region so that only multiplication operations were evaluated. A warm-up run was executed before timing, and each test was repeated ten times, with the median runtime reported to reduce variability.

Pure Python implementations used explicit loops for element-wise multiplication and triple nested loops for true matrix multiplication, while NumPy implementations relied on vectorized operations and the matrix multiplication operator (@). NumPy executes these operations using optimized compiled numerical routines designed for efficient array processing, which minimizes interpreter overhead and improves performance scalability [2], [3]. Numerical correctness between implementations was verified using np.allclose() to ensure equivalent results within floating-point tolerance.

Results and analysis

The experimental results show that performance differences between pure Python and NumPy increase significantly as problem size grows. For element-wise vector multiplication, both implementations exhibited similar runtimes at very small sizes, but the performance gap expanded steadily with larger vectors. The measured speedup increased from approximately 1× at size 10 to about 16× at size 500, indicating that NumPy's vectorized operations become increasingly advantageous as the number of loop iterations grows. While Python execution time increased gradually, NumPy runtimes remained nearly constant due to optimized numerical execution.

```
...
    VECTOR MULTIPLICATION TIMES (median seconds)
    Size 10: Python = 0.000001, NumPy = 0.000001, Speedup = 1.02x
    Size 50: Python = 0.000002, NumPy = 0.000001, Speedup = 1.75x
    Size 100: Python = 0.000005, NumPy = 0.000001, Speedup = 3.70x
    Size 200: Python = 0.000008, NumPy = 0.000001, Speedup = 6.06x
    Size 500: Python = 0.000023, NumPy = 0.000001, Speedup = 16.06x
```

Figure 3: Rundown results for 1-D arrays

The contrast became much more pronounced in true matrix multiplication. Because matrix multiplication has cubic computational complexity, the pure Python implementation using triple nested loops showed rapid growth in runtime, reaching over 19 seconds at size 500×500. In comparison, NumPy completed the same operation in a fraction of a second, producing speedups exceeding 1000× for larger matrices. This dramatic difference demonstrates how interpreter overhead accumulates when large numbers of arithmetic operations are executed in Python loops, whereas NumPy delegates the computation to optimized compiled routines.

```
.
  MATRIX MULTIPLICATION TIMES (median seconds)
  Size 10x10: Python = 0.000145, NumPy = 0.000004, Speedup = 34.06x
  Size 50x50: Python = 0.014488, NumPy = 0.000026, Speedup = 554.38x
  Size 100x100: Python = 0.127852, NumPy = 0.000115, Speedup = 1112.26x
  Size 200x200: Python = 0.936503, NumPy = 0.007998, Speedup = 117.09x
  Size 500x500: Python = 19.172705, NumPy = 0.015944, Speedup = 1202.50x
```

Figure 4: Rundown results for 2-D arrays

NumPy achieves higher performance because most numerical operations run inside compiled routines designed for efficient array processing. The NumPy array structure enables contiguous memory storage and vectorized operations, which significantly reduce execution overhead compared to Python lists and loops [2]. Additionally, NumPy provides built-in support for vectorized mathematical operations and matrix multiplication through optimized numerical libraries, allowing large datasets to be processed efficiently as input size increases [3]. In contrast, pure Python loops introduce overhead from dynamic type checking, bytecode execution, and non-contiguous memory access, which limits performance scalability.

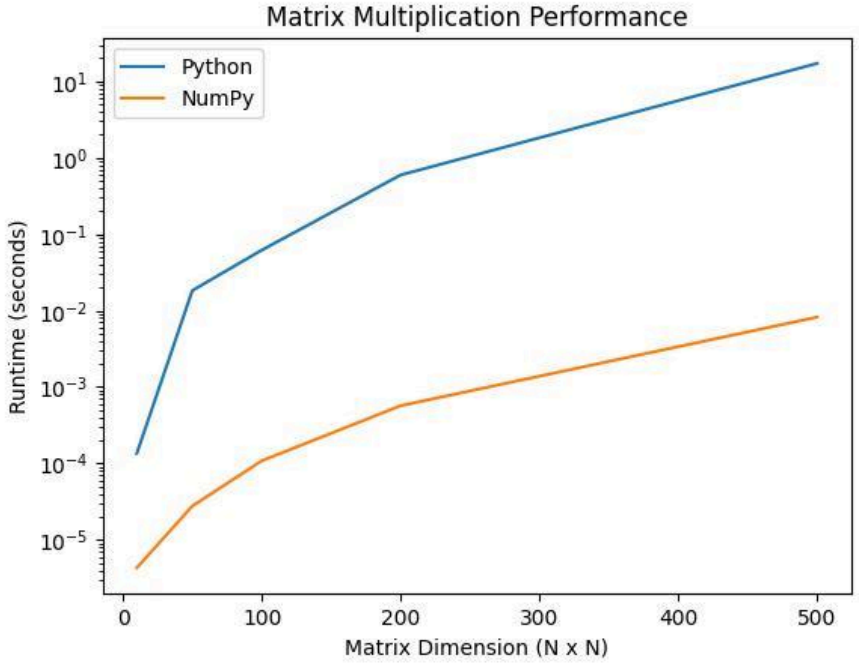

Figure 5: Runtime Graph for Python and Numpy

Interpretation of performance trends

As mentioned earlier, the most important factors relevant to this experiment are the size of the arrays whose product will be calculated and the execution time, which is directly affected by the array size. According to an article dedicated to the efficiency of Python using NumPy and Cython, NumPy allows for fast operations on multidimensional arrays, linear algebra, Fourier transforms, and tree structures.[5] Furthermore, NumPy excels at efficiently grouping multiple values within arrays, resulting in faster operations compared to pure Python, which requires iterating through each value individually.[6] This optimization makes NumPy particularly well-suited for larger datasets, where its ability to operate efficiently without the need for explicit loops offers remarkable performance gains. By focusing on test cases with a wide range of data sizes, we can obtain more distinct and consistent results. In this way, we more effectively evaluate the relative performance of NumPy versus pure Python across different data scales.

The results obtained from the experiment show that both methods operate in similar times when calculating the product of two one-dimensional arrays, although when increasing the size of the array, NumPy was faster. However, when evaluating the execution time when calculating the product

of two two-dimensional arrays, it is evident that the product times calculated by NumPy turn out to be much more efficient.

Conclusions and lessons learned

The experiment results reveal that while pure Python and NumPy can have similar times when computing the product of one-dimensional arrays, NumPy shows noticeably higher efficiency when working with two-dimensional arrays. NumPy's ability to perform operations on arrays efficiently, without the need to use explicit iterations, stands out especially on larger data sets. This study reinforces the idea that NumPy is an invaluable tool for numerical and linear algebra operations, offering consistent and efficient performance even on large arrays, in contrast to purely Python operations, which experience exponential growth in execution time as array size increases.

The experiment results reveal that while pure Python and NumPy can have similar times when computing the product of one-dimensional arrays, NumPy shows noticeably higher efficiency when working with two-dimensional arrays. NumPy's ability to perform operations on arrays efficiently, without the need to use explicit iterations, stands out especially on larger data sets. This study reinforces the idea that NumPy is an invaluable tool for numerical and linear algebra operations, offering consistent and efficient performance even on large arrays, in contrast to purely Python operations, which experience exponential growth in execution time as array size increases.

Group Collaboration

- Claudia: In charge of writing the code with some help from the rest of the team
- Manuel: In charge of debugging and testing code, writing assignment purpose, key hypothesis and Experimental design choices.
- Carlos: In charge of the measurement methodology, result and analysis, and video editing.
- Emmanuel: In charge of writing code, interpretations of performance trends, conclusions, and lessons learned sections.

References:

[1] F. Ke and L. Chen, “Research on High-Dimensional Matrix Multiplication in Machine Learning for MEC,” in *2024 10th International Conference on Computer and Communications (ICCC)*, Chengdu, China, 2024, pp. 85–91, doi: 10.1109/ICCC62609.2024.10942148.

[2] S. van der Walt, S. C. Colbert, and G. Varoquaux, “The NumPy Array: A Structure for Efficient Numerical Computation,” Computing in Science & Engineering, vol. 13, no. 2, pp. 22–30, Mar.–Apr. 2011.

[3] NumPy Developers, “NumPy Reference Guide,” NumPy Documentation. [Online]. Available: https://numpy.org/doc/

[4] “Listas (arreglos o vectores) en Python: Uso y creación,” www.programarya.com. https://www.programarya.com/Cursos/Python/estructuras-de-datos/listas

[5] S. G. de Sierra, “Reproducibilidad: ¿qué es y por qué importa?,” Sebastián Garrido de Sierra. https://www.datacrunchers.mx/blog/reproducibilidad-que-es-por-que-importa#:~:text=%22la%20reproducibilidad%20es%20obtener%20resultados (accessed Feb. 26, 2024).

[6] Morra, Gabriele (2018). [Lecture Notes in Earth System Sciences] Pythonic Geodynamics || Fast Python: NumPy and Cython. , 10.1007/978-3-319-55682-6(Chapter 3), 35–60. doi:10.1007/978-3-319-55682-6_3